

Day04 Part B-1 学习文档 v1.0 : Runtime 如何思考 (How Runtime Thinks) - Runtime State

本文是《从零实现 Agent Runtime》学习阶段的 Day04 Part B-1 正式学习文档。

Day04 Part A 已经建立了一个关键世界观：Prompt 不是 Context，Context 是 Runtime State 的一次 Projection。Part B-1 继续追问：如果 Context Builder 要投影 Runtime State，那么 Runtime 到底维护着一个怎样的世界？

与上一节的联系

Part A 的核心结论是：

Prompt 只是 Runtime State 在某一时刻的投影，不是 Agent 能力的全部来源。

但是这里会立刻出现一个新问题：

如果 Prompt 来自 Runtime State，那么 Runtime State 到底是什么？

Part B-1 的核心结论是：

Runtime 本质上是在维护一个不断变化的世界状态。Agent Loop 的每一步，都不是为了单纯生成一句回答，而是在推动 Runtime State 从一个状态迁移到下一个状态。

也就是说：

Conversation = Runtime 发生过什么
Runtime State = Runtime 认为当前世界是什么样
Context = Runtime State 被投影给 LLM 的结果
Prompt = Context 在 Provider Request 中的文本化表现之一

目录

- 为什么 Runtime 一定需要 State
- 什么是 Runtime State
- State 与 Conversation 的区别
- Runtime State 包含哪些内容
- Runtime State 是一棵 State Tree
- State 之间会互相影响
- State 不只是数据，而是系统阶段
- 谁会修改 Runtime State
- Runtime 本质上是 State Machine
- 哪些 State 会持久化，哪些不会
- Conversation、Event Log 与 Checkpoint 如何用于排查和恢复
- Runtime State 与 React State 的类比
- Part B-1 核心结论
- 下一 Part 学习计划

写书 TODO

写书素材

本 Part 核心认知升级

1. 为什么 Runtime 一定需要 State

如果只是一个最简单的 LLM Demo，流程看起来像这样：

```
graph TD
    A[User Prompt] --> B[LLM]
    B --> C[Answer]
```

这时似乎不需要 Runtime State。

但真正的 Agent 不是一次性问答，而是连续运行的软件。比如用户说：

帮我修复整个项目。

Runtime 可能会经历：

```
graph TD
    A[读取项目] --> B[分析目录]
    B --> C[读取 package.json]
    C --> D[搜索报错]
    D --> E[修改文件]
    E --> F[运行测试]
    F --> G[再次修改]
    G --> H[提交 git]
```

在这个过程中，Runtime 必须知道：

- 当前任务是什么；
- 已经做到哪一步；
- 读过哪些文件；
- 当前有哪些错误；
- 哪个工具正在运行；
- 计划是否已经生成；
- 测试是否通过；
- 用户有哪些约束。

如果没有 State，每一轮调用 LLM 都只能重新开始。

所以：

Agent 是连续运行的软件。连续运行的软件，一定需要 State。

2. 什么是 Runtime State

抽象地说，Runtime State 是程序运行时的数据。

但这个定义太薄，不足以理解 Agent Runtime。

更有帮助的说法是：

Runtime State 是 Runtime 对整个世界的当前认知。

以 Cursor 为例，它不是一段 Prompt，而更像一个程序员。一个程序员在修 bug 时，脑子里会有很多当前状态：

```
今天要修 login bug
现在正在改 login.ts
Git 有三个修改
还有两个 TODO
刚才执行过 npm test
测试失败
失败原因是 Type Error
用户要求不要修改 API
```

这些东西不是 Prompt，也不只是聊天记录，而是 Runtime 当前认为世界所处的状态。

所以 Runtime State 描述的不是“用户说过什么”，而是：

```
世界现在是什么样
Runtime 正在做什么
下一步应该基于什么继续
```

3. State 与 Conversation 的区别

Conversation 和 Runtime State 很容易混淆。

Conversation 保存的是：

```
发生过什么
```

例如：

```
User:
修复 bug
```

```
Assistant:
好的，我先读取文件
```

```
ToolCall:
read_file(login.ts)
```

```
ToolResult:
返回文件内容
```

```
Assistant:
发现类型错误
```

这是历史，是事件时间线。

Runtime State 保存的是：

现在是什么状态

例如：

Workspace 已加载
当前文件：login.ts
当前任务：Fix Login
Tool：Busy
Git Dirty：true
Plan：Step 4/8
Diagnostics：5 Errors

可以这样概括：

Conversation = Event Log
Runtime State = Current Snapshot

一个记录历史，一个描述现在。

这也是为什么 Conversation 不等于 Context。Context Builder 不会把整个历史原封不动交给 LLM，而是会基于当前 State 选择、裁剪、排序、转换，生成这一轮 LLM 需要看到的 Context。

4. Runtime State 包含哪些内容

一个工业级 Runtime 的 State 通常不是一个字段，而是一整套状态模块：

```
RuntimeState
├── Conversation
├── Summary
├── Memory
├── Planner
├── Workflow
├── Current Task
├── Workspace
├── Files
├── Diagnostics
├── Tool Registry
├── Tool Results
├── Provider
├── Permission
├── Metadata
├── Variables
├── User Profile
├── Checkpoints
├── Context Policy
└── Metrics
```

这不是固定标准，不同 Runtime 会有不同结构。

但思想是一致的：

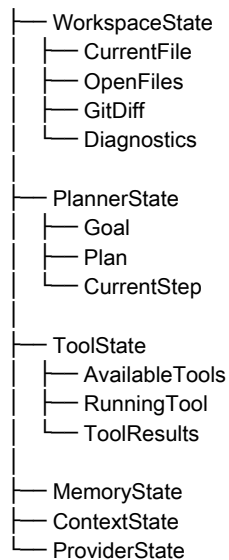
Runtime 维护的是一个完整世界，而不是一段聊天记录。

5. Runtime State 是一棵 State Tree

Runtime State 不是平面的对象，而是一棵 State Tree。

例如：

```
RuntimeState
├── ConversationState
```



以后看 LangGraph、OpenAI Agents SDK、Mastra 或其他 Runtime 的源码时，会发现它们通常都不是：

```
const state = {}
```

而更接近：

```
const runtime = {  
  conversation,  
  planner,  
  workspace,  
  memory,  
  tools,  
  provider,  
}
```

真正被维护的是一棵 Runtime State Tree。

6. State 之间会互相影响

State 不是彼此孤立的。

例如 WorkspaceState 更新：

```
WorkspaceState  
  
CurrentFile = login.ts  
Diagnostics = 5 errors
```

这可能导致 PlannerState 变化：

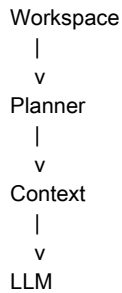
```
PlannerState  
  
Plan = Fix login.ts  
CurrentStep = Read login.ts
```

PlannerState 又会影响 ContextState：

```
ContextState  
  
Include current file  
Include diagnostics  
Include current plan
```

ContextState 又会影响 LLM Output。

所以真正的 Runtime 更像一个动态联动系统：



这和前端复杂 Form 的联动非常像。比如省份变了，城市要清空，区县也要清空，同时重新请求 cityList。

可以写成一句适合给前端工程师看的话：

Runtime State 的联动关系，本质上和复杂 Form 的联动没有区别，只不过联动对象从 Input 变成了 Planner、Tool、Memory、Context 等模块。

7. State 不只是数据，而是系统阶段

很多程序员第一次看到 State，会把它理解成几个字段：

```
{
  file: "login.ts",
  task: "fix bug"
}
```

但更准确地说：

State 描述的是系统当前所处的阶段，数据只是这个阶段的表现形式。

React 中的 loading = true 不只是一个 boolean，而是表示整个页面进入 Loading State。

Runtime 也是一样：

Planning

不只是一个字符串，而是表示整个 Runtime 正在制定计划。

Tool Calling

不只是一个枚举值，而是表示 Runtime 正在等待工具调用完成。

所以：

State 不是几个字段，而是系统当前所处的世界状态。

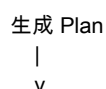
8. 谁会修改 Runtime State

Runtime State 的修改来源很多，不只是用户。

用户会修改 State



LLM 会修改 State



PlannerState 更新

Tool 会修改 State

Read File
|
v
WorkspaceState 更新
ToolResult 更新

Runtime 自己会修改 State

Retry
|
v
Metadata 更新
Retry Count 更新

Provider 也会影响 State

LLM Usage
|
v
Token Usage 更新
Cost Metrics 更新

所以 Runtime State 不是某个模块单独维护的，而是整个 Runtime 在 Agent Loop 中共同维护的。

9. Runtime 本质上是 State Machine

很多人以为 Runtime 是：

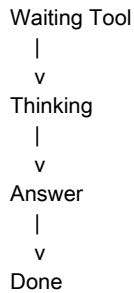
```
while (true) {  
  callLLM()  
}
```

但真正的 Runtime 更像：

Current State
|
v
Action
|
v
Next State
|
v
Action
|
v
Next State

例如：

Idle
|
v
User Message
|
v
Planning
|
v
Tool Calling
|
v



每一步都是状态迁移。

因此：

真正驱动 Runtime 的不是 Prompt，而是 State。

10. 哪些 State 会持久化，哪些不会

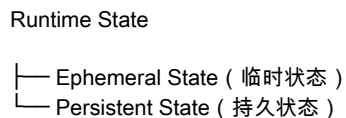
一个重要问题是：

Runtime State 每次变化都要落库吗？

答案是：

不是。Runtime State 并不是每一次变化都落库。绝大多数 State 是内存态，只有部分 State 会在特定时机持久化。

可以把 Runtime State 分成两类：



临时状态通常只存在于运行过程中：

- Current Plan
- Current Tool
- Current Retry Count
- Current Step
- Current Streaming Buffer
- Current Tool Result
- Current Context
- Current Token Usage

这些状态变化很快，通常保存在内存里，不会每次都写数据库。

持久状态则需要保存：

- Conversation
- Memory
- Summary
- Checkpoint
- Workflow Snapshot

注意，不是“最终 State 整体落库”，而是：

Runtime State 里面有些字段天然就是持久态，有些字段天然就是临时态。

Context 和 Prompt 一般不单独落库，因为它们是 Runtime State 的 Projection，不是 Runtime State 本身。

11. Conversation、Event Log 与 Checkpoint 如何用于排查和恢复

如果中间临时 State 不落库，出了问题如何排查？

工业 Runtime 通常靠三类信息。

第一类：Conversation

Conversation 是 Runtime 的事件时间线，记录已经发生的事实：

```
User:
修复登录 Bug

Assistant:
准备分析项目

ToolCall:
read_file(login.ts)

ToolResult:
返回文件内容

Assistant:
发现类型错误
```

这些记录可以用于回放、调试和总结。

第二类：Runtime Event Log

真正工业 Runtime 还会记录运行日志：

```
Planning Start
Tool Selected: ReadFile
Retry: 2
LLM Cost: $0.03
Provider Latency: 850ms
```

这些日志不是 State，而是 Runtime Event。

第三类：Checkpoint

如果 Agent 已经跑了二十分钟，服务器突然挂了，就需要 Checkpoint：

```
Checkpoint

Conversation
Planner
Variables
Workflow
CurrentStep
```

恢复时可以从 Checkpoint 继续执行，而不是从头开始。

这也是 LangGraph、Temporal、Durable Execution 等系统复杂性的来源之一：它们要解决的不只是调用模型，而是 Runtime Crash Recovery。

12. Runtime State 与 React State 的类比

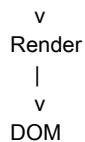
这一节里最有价值的类比来自 React。

React 的经典公式是：

```
UI = Render(State)
```

也就是：

```
State
|
```



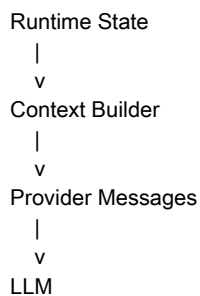
Agent Runtime 也可以写成类似公式：

LLM Input = ContextBuilder(Runtime State)

或者更准确地说：

Provider Messages = Projection(Runtime State)

结构上就是：



可以做一张对照表：

React	Agent Runtime	State	Runtime State	Render	Context Builder	DOM	Provider Messages	Browser	LLM	User Event	User / Tool Event	setState	Runtime Update
-------	---------------	-------	---------------	--------	-----------------	-----	-------------------	---------	-----	------------	-------------------	----------	----------------

这句话非常适合作为 Day04 的核心表达：

React 的 Render 根据 State 渲染 UI；Agent Runtime 的 Context Builder 根据 Runtime State 渲染 LLM 所看到的世界。

13. Part B-1 核心结论

如果 Part A 的一句话是：

Prompt 是 Runtime State 的一次 Projection。

那么 Part B-1 的一句话就是：

Runtime 本质上是在维护一个不断变化的世界状态 (Runtime State)，Agent Loop 的每一步都只是推动这个世界进入下一个状态。

这一节完成之后，Day04 的逻辑已经变得非常清晰：

Day04 Part A

Prompt ≠ Context
Context ≠ Conversation
Runtime State -> Projection -> Context



Day04 Part B-1

Runtime 为什么必须维护 State
State ≠ Conversation
State 是一棵 State Tree
State 会互相影响
Runtime = State Machine

14. 下一 Part 学习计划

下一节建议新增为：

Day04 Part B-2 : Runtime State Lifecycle —— 一个 Agent 的状态是如何诞生、变化、持久化和消亡的？

它将回答：

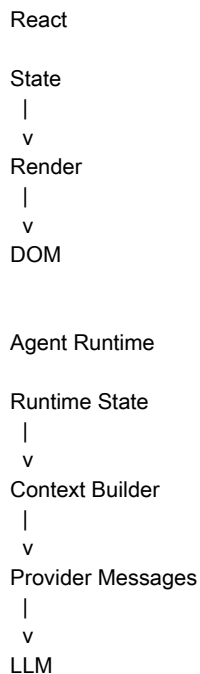
- Runtime State 是什么时候创建的；
- 一个 User Message 如何变成 Runtime State；
- Runtime State 在 Agent Loop 中如何不断演化；
- Conversation、Summary、Memory、Checkpoint 分别在什么时候生成；
- 哪些 State 是短生命周期，哪些 State 会跨 Session；
- Runtime 结束以后哪些 State 会消失，哪些会留下；
- 为什么 Durable Execution 一定需要 Checkpoint。

Part B-2 是承上启下的一节。它解释 State 从哪里来、怎样变化、什么时候消失，再为 Part C 的 Context Builder 做铺垫。

15. 写书 TODO

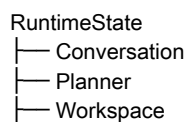
TODO 1：增加 React 与 Runtime 对照图

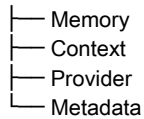
新增一张可以贯穿 Day04、Day05、Context Builder、Provider Adapter 的重要插图：



TODO 2：新增 Runtime State Tree 插图

建议增加：





以后讲源码时可以直接引用。

TODO 3：增加“给前端工程师看的 Runtime State”小节

内容可以包括：

- Form 联动；
- Redux；
- React Render；
- Vue 响应式；
- Runtime State；
- Context Builder 与 Render 的类比。

TODO 4：增加“事件 (Event) 与状态 (State)”专题

Conversation 记录 Event，Runtime State 描述 Current Snapshot。

这个区别是软件工程经典知识，也是理解 Agent Runtime 的基础。

16. 写书素材

素材一

Prompt 不是 Runtime 的输入，Runtime State 才是。Prompt 只是 Runtime State 在某一时刻的投影 (Projection)。

素材二

Conversation 记录的是发生过什么 (Event)，Runtime State 描述的是现在是什么 (Current World)。

素材三

React 的 Render 根据 State 渲染 UI；Agent Runtime 的 Context Builder 根据 Runtime State 渲染 LLM 所看到的世界。

素材四

Runtime 并不是在不断生成 Prompt，而是在不断维护一个不断演化的世界状态 (Runtime State)。

素材五

Agent Loop 的每一步，都不是为了生成一句回答，而是在推动 Runtime State 从一个世界演化到下一个世界。

17. 本 Part 核心认知升级

第一层：概念升级

过去理解为：

Prompt

|
v
LLM
|
v
Answer

现在理解为：

Runtime State
|
v
Context Builder
|
v
Prompt / Provider Messages
|
v
LLM

Prompt 退居二线，真正的主角变成 Runtime State。

第二层：架构升级

过去认为：

Runtime = while(true) { callLLM() }

现在变成：

Runtime = 维护 Runtime State + 推动 Runtime State 演化

Runtime 不再只是调用模型的循环，而是一个持续演化的状态系统。

第三层：思维升级

前端有：

UI = Render(State)

Agent Runtime 有：

LLM Input = ContextBuilder(Runtime State)

更准确地说：

Provider Messages = Projection(Runtime State)

这就是 Day04 Part A 与 Part B-1 合起来之后最重要的统一公式。